

Clase 10

QA, CI & CD

IIC2513 - Tecnologías y Aplicaciones Web

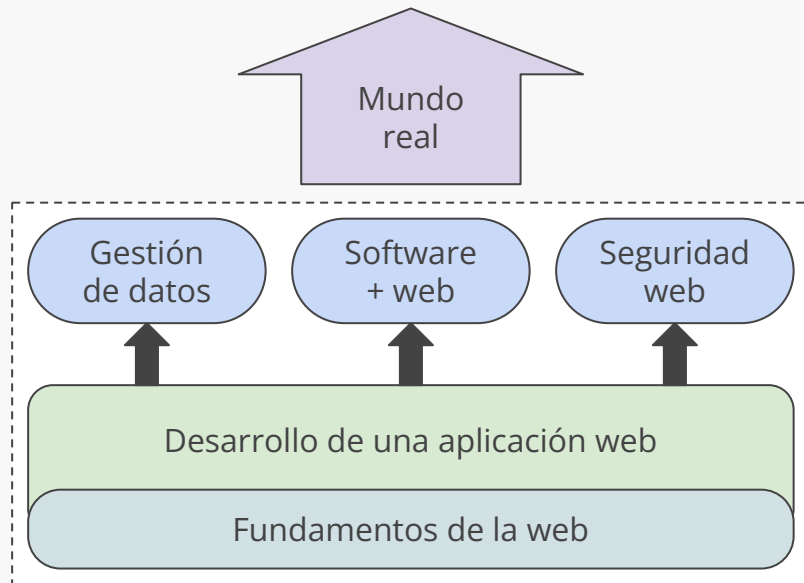
Antonio Ossa Guerra
aaossa@ing.puc.cl

1. Hay ayudantía mañana
2. Tarea 2 desde mañana (ver 1)
3. Recuerden responder la ETC

Anuncios

Contenidos del curso

- (1) Fundamentos de la web
- (2) Desarrollo de una app web
- (3) Gestión de datos
- (4) Software + web
- (5) Seguridad web
- (6) El mundo real

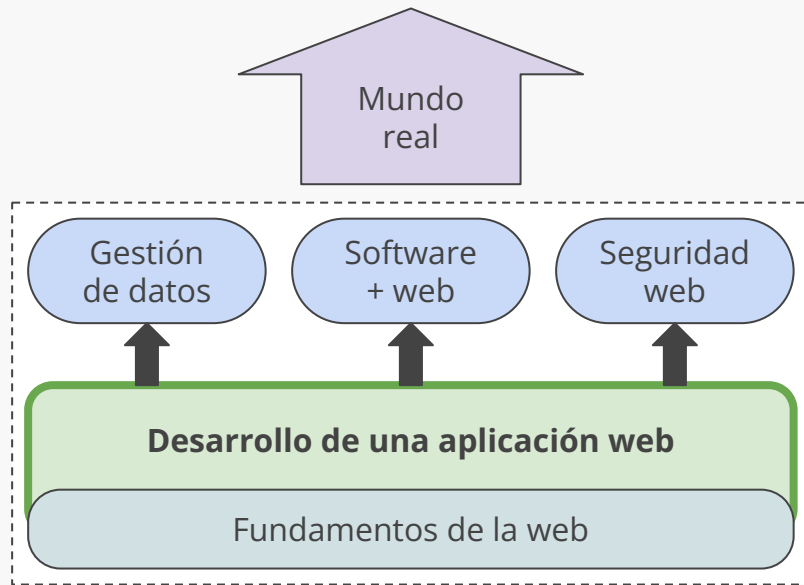


Contenidos del curso

Desarrollo de una aplicación web

Veremos herramientas como React, Node.js, Koa y Express, y técnicas como CSS avanzado y el uso de *Design Systems*

... para **desarrollar** aplicaciones modernas tanto en el lado del **cliente** (front-end) como en el lado del **servidor** (back-end)

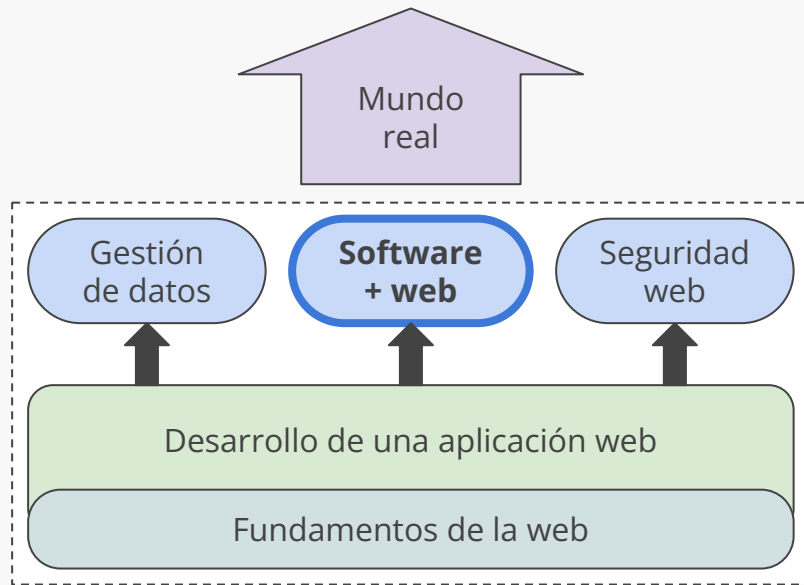


Contenidos del curso

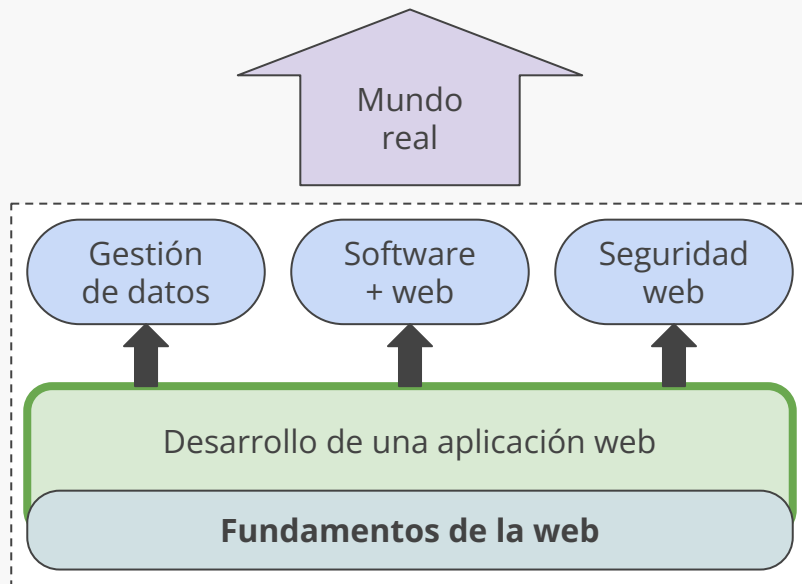
Software + web

Hablaremos sobre elementos que apuntan a expandir y **mejorar la experiencia de desarrollo web** tales como *testing* y *websockets*...

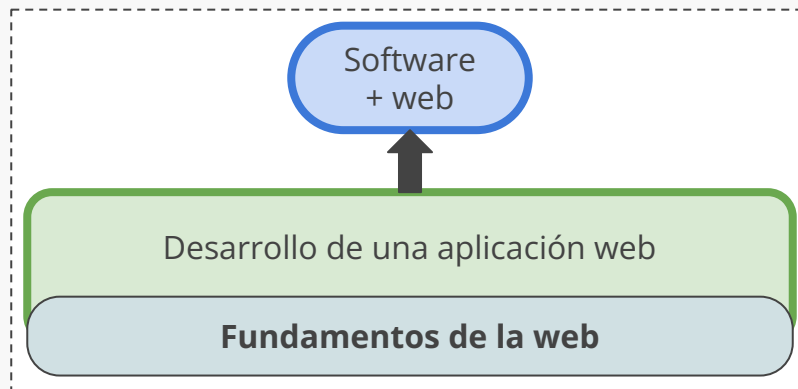
... para reconocer e identificar tecnologías de vanguardia y que permiten **buenas prácticas en desarrollo web**



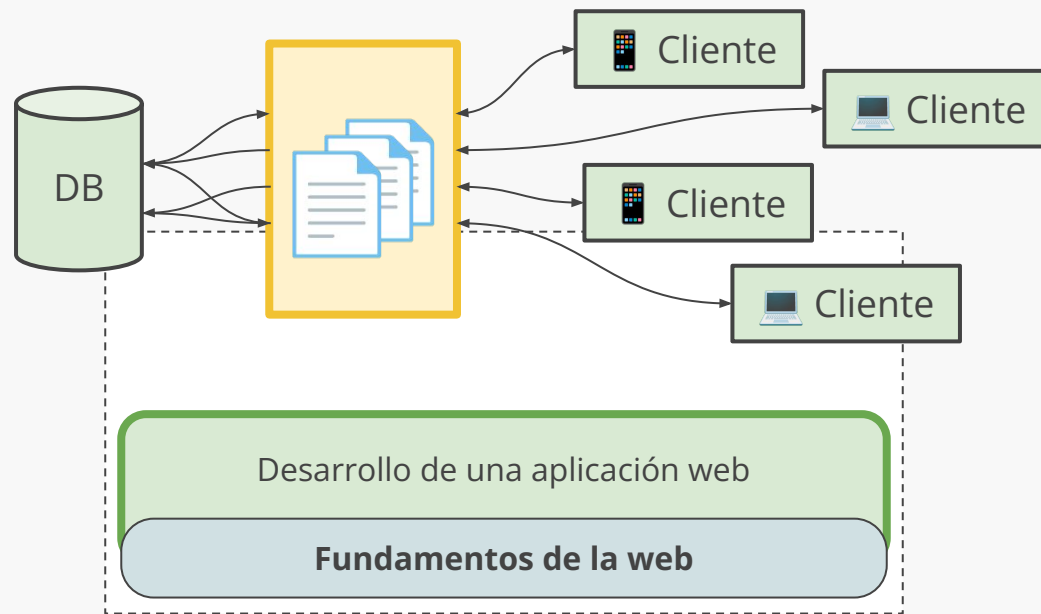
Contenidos del curso



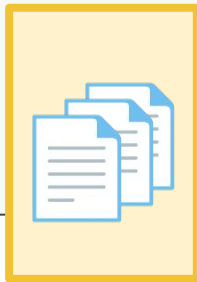
Contenidos del curso



Motivación



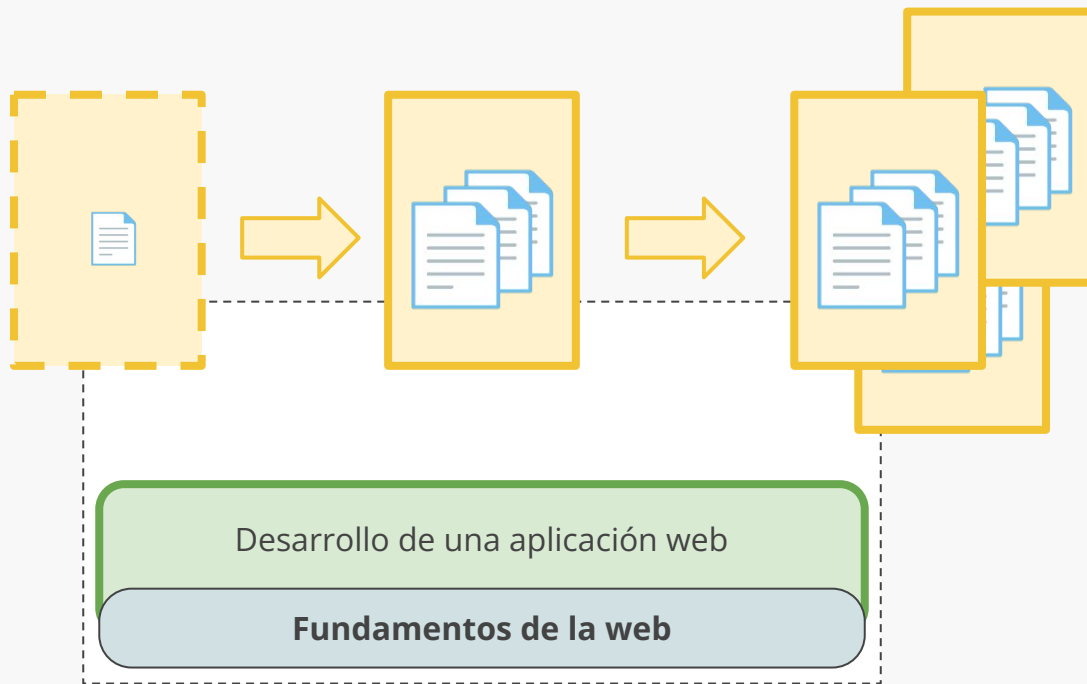
Motivación



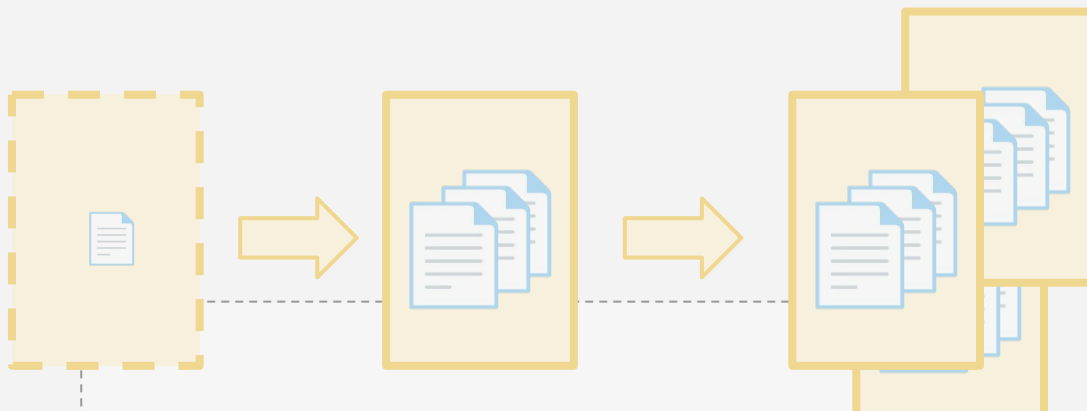
Desarrollo de una aplicación web

Fundamentos de la web

Motivación



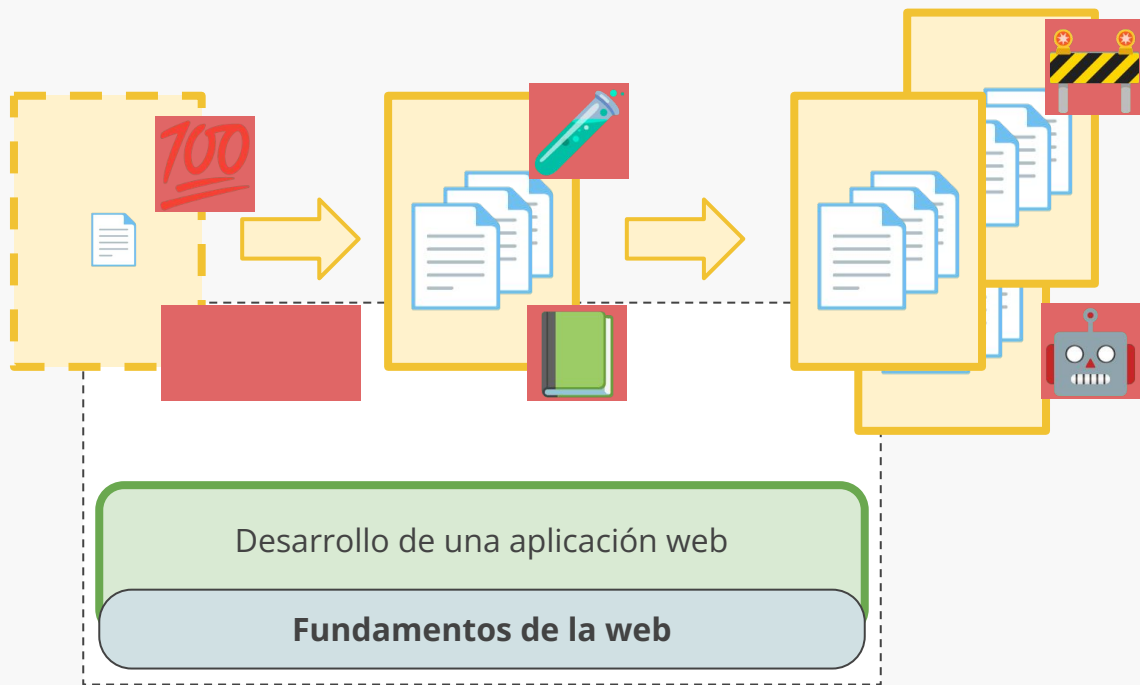
Motivación



Pregunta#¿Cómo desarrollar “bien”?

Fundamentos de la web

Motivación



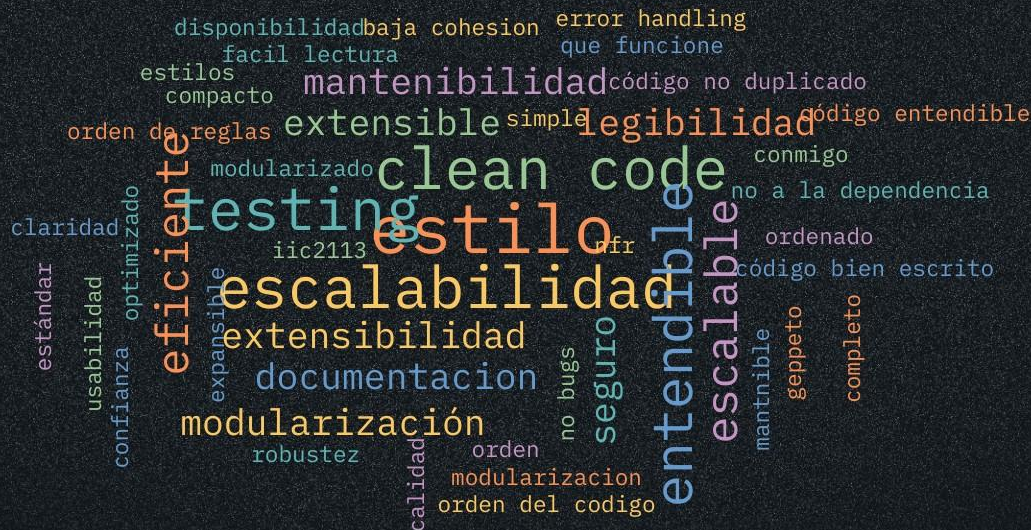
Menti*#Hablemos de calidad de software*

¿Cómo te fue con el Control 2? 🙄

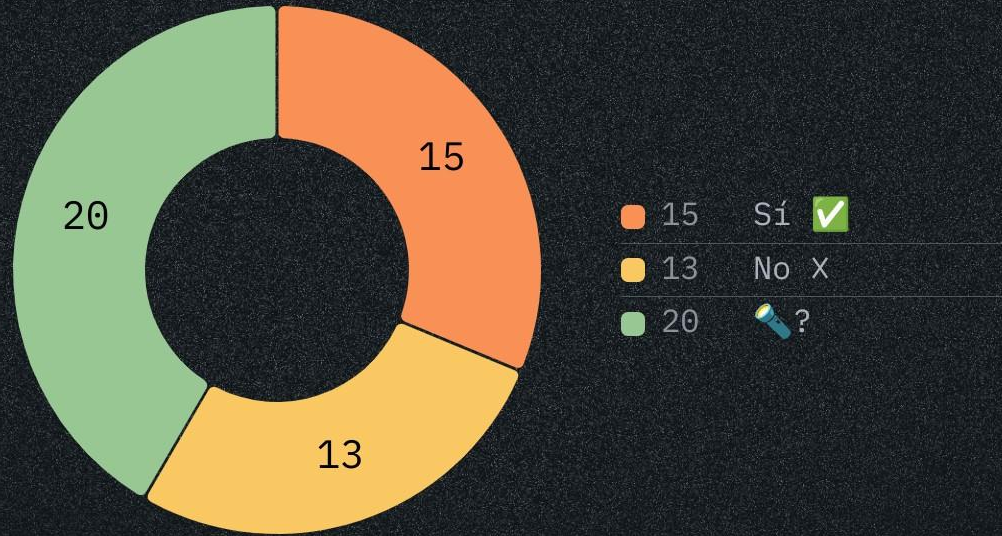


Hablemos sobre calidad de código

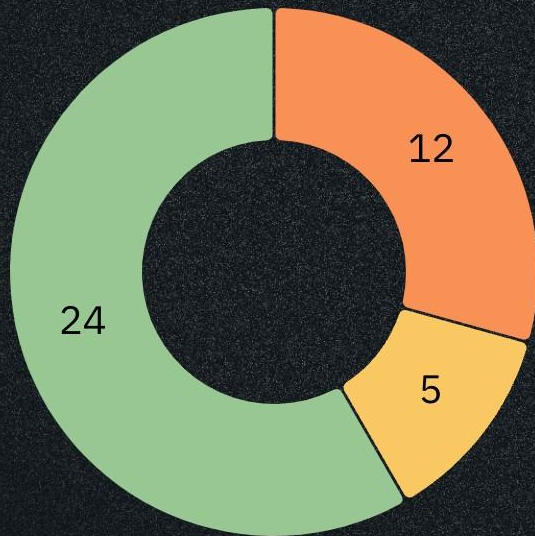
79 Respuestas



¿Has usado alguna vez un linter?



¿Has hecho alguna PR en algún curso?



- 12 Sí ✓
- 5 No X
- 24 ¿Qué es una PR? ?

¡Gracias por responder!

Mañana hay ayudantía, les
servirá para la Tarea 2

Agenda

✨ Calidad de software

QA : Aseguramiento de calidad

CI: *Continuous Integration*

CD: *Continuous Deployment*

Calidad de software

Área de estudio o práctica que describe los **atributos deseables** de un **producto de software**

Atributos deseables:

- Que haga lo debe hacer
- Que lo haga sin errores

¿Por qué es importante?

Siempre hay que recordar que
**el software que construimos suele tener un
impacto en los usuarios y en los interesados**

- **Riesgo:** problemas en software pueden tener consecuencias con distinto nivel de gravedad (hablaremos de esto más adelante en el curso)
- **Costo:** software de buena calidad puede reducir costos, y uno de mala calidad puede costar bastante caro

¿Por qué es importante?



¿Por qué es importante?

- Una de las responsabilidades de quien desarrolla código es estar consciente de que **lo que se desarrolla tendrá un impacto** en otros y en el entorno
- **El software no existe en existe en un vacío**, sino que lo hace dentro de un contexto sobre el que influye
- **Hay deberes** legales, éticos y morales en juego

Seamos conscientes del **impacto de construir software** y de la **responsabilidad** que tenemos como desarrolladores

Algunas responsabilidades

Nuestra responsabilidad incluye **minimizar riesgos**:

- Privacidad
- Seguridad de los datos
- Seguridad de las personas
- Confiabilidad
- Facilidad de uso

Las buenas prácticas existen para reducir riesgo de problemas

Impacto práctico

Baja calidad

- Cybermonday - cyberday
- ~~PortalUC~~
- Therac-25
- Toyota y Tesla
- Más ejemplos

Alta calidad

- Rover Perseverance de la NASA
- Google Suite (😓)
- ¿Alguno más?

Impacto práctico

Baja calidad

- Cybermonday - cyberday
- ~~PortalUC~~
- Therac-25

Alta calidad

- Rover Perseverance de la NASA
- Google Suite (😓)
- ¿Alguno más?

Pregunta# ¿Por qué es más fácil pensar en software de baja calidad?

Impacto práctico

Baja calidad

- Cybermonday - cyberday
- ~~PortalUC~~
- Therac-25

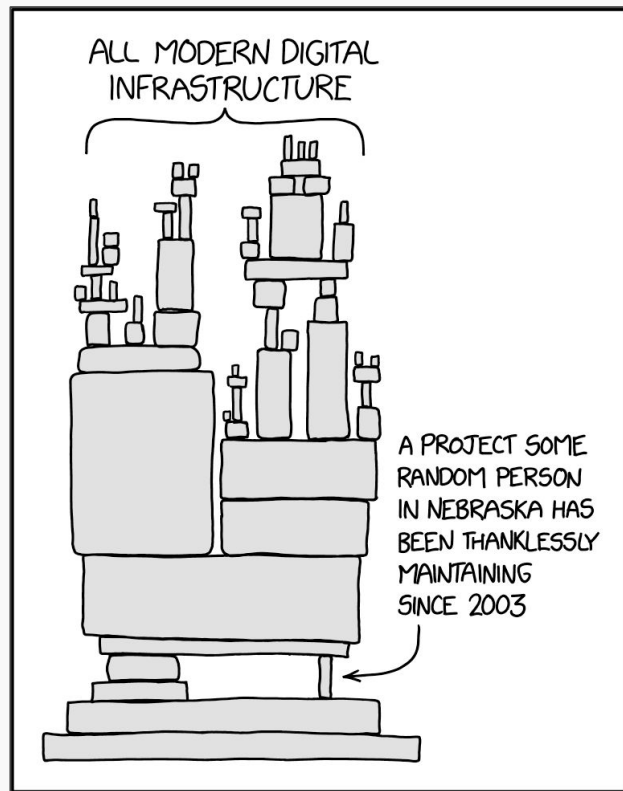
Alta calidad

- Rover Perseverance de la NASA
- Google Suite (😓)
- ¿Alguno más?

Pregunta# ¿Dónde y cómo se aprender a construir software de alta calidad?

¿“Buenas prácticas” y “buen código”?

La práctica en los **detalles** permite al profesional mayor dominio y confianza en su trabajo. Por otra parte, un fallo en construcción genera un desencanto por el trabajo en general (*Robert C. Martin, 2008*)



How one programmer broke the internet by deleting a tiny piece of code:

Artículo sobre qué ocurrió detrás del “[npm left-pad incident](#)”



Free Newsletters

QUARTZ

Editions



HOME

LATEST

BUSINESS NEWS

MONEY & MARKETS

TECH & INNOVATION

A.I.

LIFESTYLE

LEADERSHIP

EMAILS

PODCASTS

TECH & INNOVATION

How one programmer broke the internet by deleting a tiny piece of code

A man in disrupted web development around the world by deleting 11 lines of code

By **Keith Collins** Published March 27, 2016

```
1 module.exports = leftpad;
2 function leftpad(str, len, ch) {
3   str = String(str);
4   var i = -1;
5   if (!ch && ch !== 0) ch = ' ';
6   len = len - str.length;
7   while (++i < len) {
8     str = ch + str;
9   }
10  return str;
11 }
12
13
14
15
16
17
18
19
20
```


Agenda

Calidad de software



QA: Aseguramiento de calidad

CI: *Continuous Integration*

CD: *Continuous Deployment*

Aseguramiento de Calidad (*Quality Assurance*, QA)

Conjunto de **prácticas y procesos** que nos permiten **asegurar** una **calidad de software adecuada**

Algunas ideas:

- **Cubrir las características mencionadas** en mayor o menor medida
- **Partir por lo más crítico...** ¿Qué sería lo más crítico?

Aseguramiento de Calidad (*Quality Assurance*, QA)

Conjunto de **prácticas y procesos** que nos permiten **asegurar** una **calidad de software adecuada**

Algunas ideas:

Pregunta# ¿Existe el software sin errores?

Aseguramiento de Calidad (*Quality Assurance*, QA)

Pregunta#¿Existe el software sin errores?

No, el software no es infalible

Toda pieza de software es hecha por humanos,
y los humanos cometemos errores

¿Qué podemos hacer?

Lo que queremos lograr es: minimizar errores en el software que construimos, y verificar que el software funcione y sea mantenible. Para lograrlo, contamos con estas técnicas:

Testing



Análisis estático



Code review



Testing



Diseño de pruebas
automatizadas con el
objetivo de someter al
software a diferentes
escenarios

Testing

Probar que la funcionalidad cumple lo especificado y también que es robusta frente a potenciales errores. Bastante común en librerías de código abierto, en piezas de software usadas en producción... aunque poco común en proyectos personales

(Más sobre esto la [futura clase de Testing](#) y en [IIC3745 - Testing](#))

Clase 18#*Testing*

(Más sobre esto la **futura clase de *Testing*** y en **IIC3745 - Testing**)

Análisis estático

100

Análisis que se realiza de una pieza de software mientras no está ejecutándose

Análisis estático de código

Implica **analizar el código fuente**, sin ejecutarlo (por eso es “estático”), y se suelen utilizar herramientas automatizadas. Se analiza en base a criterios pre-definidos: conjunto de reglas y/o guías de estilo. ¿Qué ganamos?

- Identificar código duplicado
- Mejorar manejo de errores
- Encontrar problemas de sintaxis
- Identificar *code smells*
- Cumplir con buenas prácticas

Linters

Son herramientas que automatizan el análisis estático de código según lo que estemos usando



ESLint

JavaScript



RuboCop

Ruby



Python

Linters - ESLint



ESLint es una herramienta que define una base de reglas para evaluar el código Javascript

- **Configurable:** qué reglas aplicar
- **Extensible:** capacidades agregadas por medio de *plug-ins*



[Team](#)

[Blog](#)

[Docs](#)

[Store](#)

[Playground](#)

[Code Explorer](#)

[Donate](#)

Find and fix problems in your JavaScript code

ESLint statically analyzes your code to quickly find problems. It is built into most text editors and you can run ESLint as part of your continuous integration pipeline.

```
 npm init @eslint/config@latest
```

[Get Started](#)

[Become a Sponsor](#)

Latest Version

 v9.24.0 on 4 Apr

Upcoming Version

 v9.25.0 on 18 Apr

GitHub - airbnb/javascript: JavaScript Style Guide:

Guía de estilo para JavaScript desarrollada por Airbnb

README Code of conduct MIT license Security

Airbnb JavaScript Style Guide() {

A mostly reasonable approach to JavaScript

Note: this guide assumes you are using [Babel](#), and requires that you use [babel-preset-airbnb](#) or the equivalent. It also assumes you are installing shims/polyfills in your app, with [airbnb-browser-shims](#) or the equivalent.

downloads 16M/month downloads 29M/month [gitter](#) [join chat](#)

This guide is available in other languages too. See [Translation](#)

Other Style Guides

- [ES5 \(Deprecated\)](#)
- [React](#)
- [CSS-in-JavaScript](#)
- [CSS & Sass](#)
- [Ruby](#)

Table of Contents

Releases

102 tags

Packages

No packages published

Contributors 500



[+ 486 contributors](#)

Deployments 9

[github-pages](#) 7 years ago

[+ 8 deployments](#)

Languages

JavaScript 100.0%

Code review



Revisión del código fuente
por parte de pares
(desarrolladores parte o
no del proyecto)

Code Review

El objetivo es poder detectar errores, asegurar el uso de buenas prácticas y eventualmente encontrar mejores formas de solucionar problemas. ¿Cómo funciona?

- El **autor** de los cambios soluciona problemas o aplica las sugerencias
- En general, se requiere que los cambios sean aprobados por los **reviewers** antes de integrarlos en el repositorio de código compartido
- El output de una *review* pueden ser sugerencias o solicitudes de cambio

¿Qué ganamos?

- Detectar errores tempranamente
- Asegurar uso de buenas prácticas y guías de estilo
- Eventualmente, encontrar mejores formas de solucionar problemas
- Aprender en base al código revisado
- Mejorar la comunicación
- Documentar el proceso de diseño y la toma de decisiones

Flujo típico

1. **Autor** implementa nueva funcionalidad y solicita *review*
2. **Reviewer** solicita cambios o entrega sugerencias
3. **Autor** realiza cambios y solicita *re-review*
4. **Reviewer** aprueba cambios
5. **Autor** integra cambios

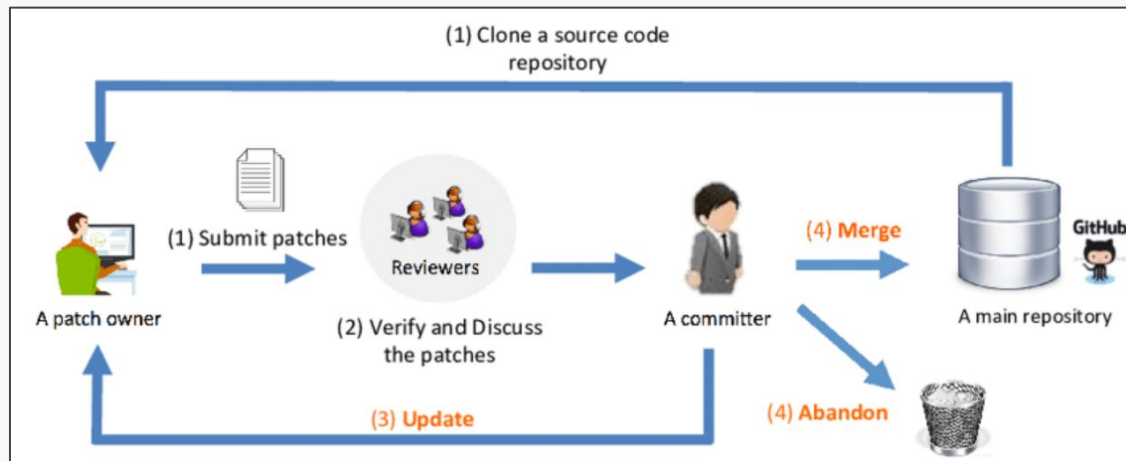


Figura: [\(PDF\) The Impact of a Low Level of Agreement Among Reviewers in a Code Review Process](#)

Algunos consejos

Como autor:

- No tomarse los comentarios de manera personal
- Actitud humilde y receptiva
- Defender posturas con argumentos

Como *reviewer*:

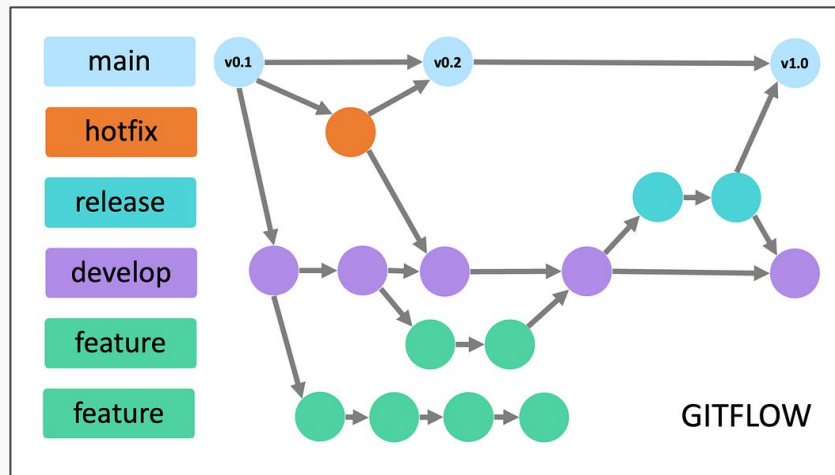
- Tener en mente que estamos dando *feedback* a una persona
- Mantener una intención positiva y constructiva
- Sugerir alternativas en lugar de “soluciones únicas”
- Utilizar argumentos al solicitar cambios
- Destacar buenas soluciones

GitHub flow

Flujo de desarrollo que aprovecha GitHub

Algunas reglas:

- Utilizar la rama **main** en GitHub
- Siempre debe estar **estable** y en condiciones de ser utilizada
- Revisión de código se realiza en **Pull Requests (PRs)**



GitHub flow: *commits*

Un *commit* es un **conjunto empaquetado de cambios**

- Incluye un mensaje y se le asigna un **identificador** (*hash*, *sha*)
- Cada conjunto de cambios debe tener un propósito
- El propósito se ve reflejado en el **mensaje de *commit***
- Sirve como **documentación** del proceso de desarrollo

Existen muchos repositorios públicos que siguen una guía de estilo y buenas prácticas para asegurar buenos mensajes de *commits*, basta con ver cualquier proyecto relativamente grande. Un gran ejemplo de commit: [My favourite Git commit | dhwthompson.com](https://github.com/dhwt/thompson/blob/master/commit/My%20favourite%20Git%20commit%20-%20dhwthompson.com)

GitHub flow: *commits*

<commit header>

Encabezado

Comunica de forma resumida el cambio realizado

<commit body>

Línea en blanco

Separa el *header* del *body*

Cuerpo del *commit*

Explicación detallada e información adicional

Convención “Angular”

Guía muy popular para escribir mensajes de *commits*, creada por Angular ([Angular](#))

3 partes en la primera línea del commit para resumir el tipo y contenido de los cambios, y luego una descripción más extensa (opcional)

`<type>(<scope>) : <summary>`

`<commit body>`

`docs: a short summary in present tense
without capitalization or ending period`

`A description of what was fixed, and why.
This description can be as detailed as
necessary and can be written with
appropriate capitalization and
punctuation`

`Fixes #34353`

Convención “Conventional Commits”

Especificación para aprovechar mensajes de *commits*, buscando que humanos y máquinas entiendan el significado de un mensaje

([Conventional Commits](#))

<type>[optional scope]: <summary>

[optional body]

[optional footer(s)]

```
fix: prevent racing of requests
```

```
Introduce a request id and a reference to  
latest request. Dismiss  
incoming responses other than from latest  
request.
```

```
Remove timeouts which were used to  
mitigate the racing issue but are  
obsolete now.
```

```
Reviewed-by: Z
```

```
Refs: #123
```

Convención “Platanus”

Especificación para construir mensajes de *commits* dentro del equipo de desarrollo de Platanus ([Platanus](#))

`tipo(contexto):` descripción

```
fix(ui/SignUpForm): validate username
availability
```

The `SignUpForm` component wasn't validating the availability of the `username` field and only displayed a `couldn't create account` error on submit.

This commit adds an asynchronous validation when typing on the field so the user will know if the username is available before completing further fields.

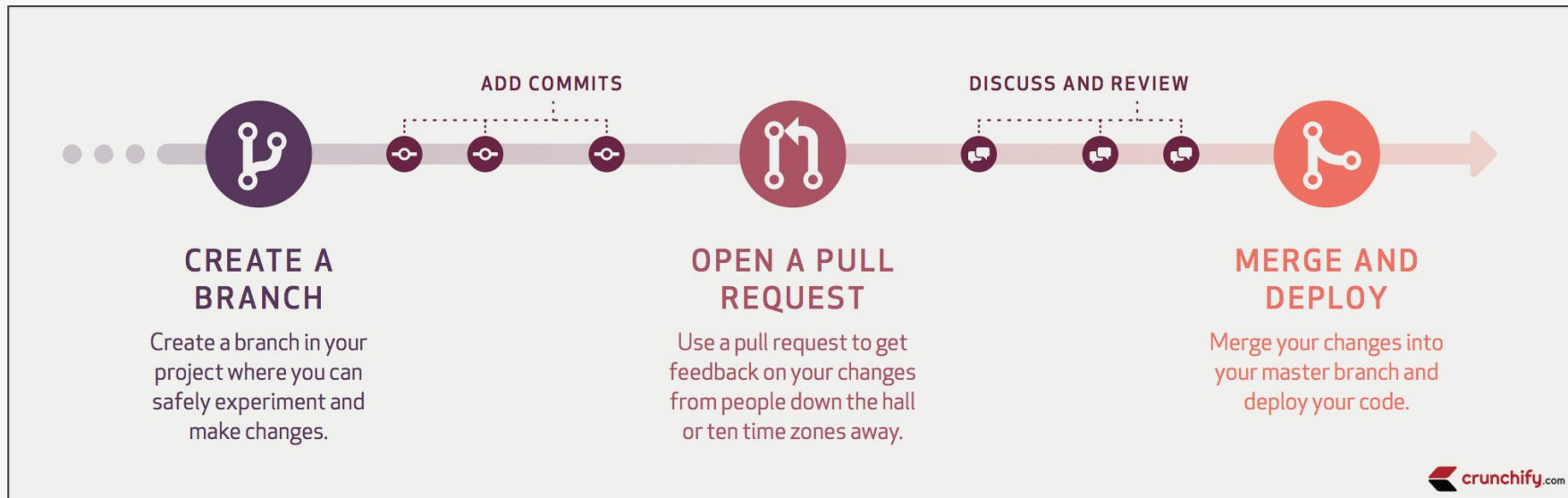
En el mundo real

En general, todo proyecto define y utiliza una convención a la hora de escribir *commits*, la que siguen sus mantenedores y contribuidores

Algunos ejemplos:

- [Commits en sequelize/sequelize](#)
- [Commits en postgres/postgres](#)
- [Commits en vuejs/vue](#)
- [Commits en pandas-dev/pandas](#)

GitHub flow: pasos a seguir



GitHub flow: pasos a seguir

1. Cada nueva *feature* debe ser implementada en una nueva *branch* (*feature branch*), creada a partir de **main**
2. Agregar *commits* con cambios asociados a dicha *feature*
3. Abrir una **Pull Request (PR)** cuando la *feature* ha sido implementada
4. La PR es **asignada a reviewers** que pueden solicitar cambios o aprobar
5. Si se solicitan cambios, se pueden agregar nuevos *commits*
6. Se repiten (4) y (5) hasta que la PR sea aprobada (o cerrada)
7. Si la PR se aprueba, se hace *merge* directamente desde GitHub

Agenda

Calidad de software

QA: Aseguramiento de calidad

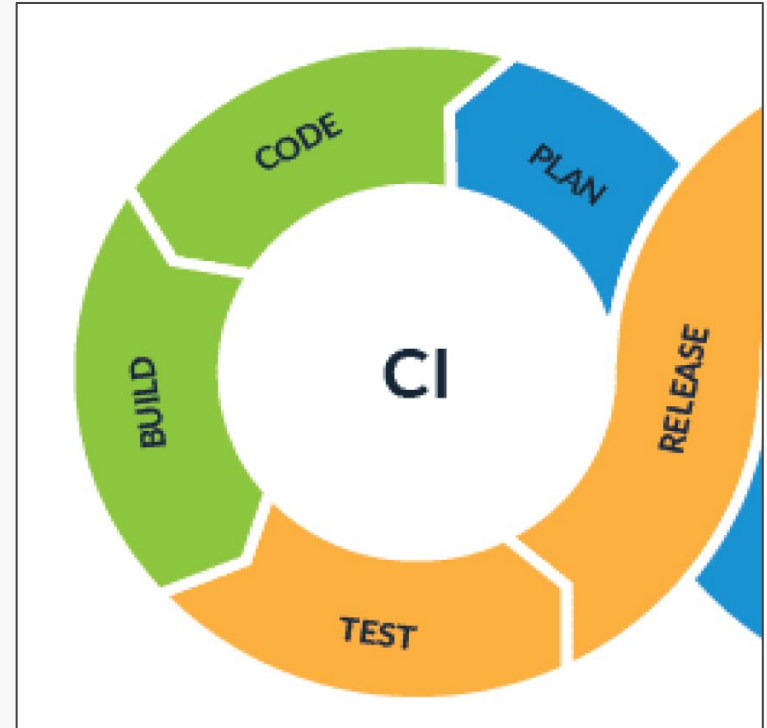


CI: *Continuous Integration*

CD: *Continuous Deployment*

Continuous Integration (CI)

- Integración Continua es una metodología que fomenta la integración de cambios en el código de manera continua, durante el desarrollo
- Busca disminuir los costos de integración y detectar fallas de forma temprana
- Es buena práctica automatizar este proceso con herramientas como:
 - Jenkins CI
 - Circle CI
 - Travis CI
 - (CloudBees) Codeship CI



Features • GitHub Actions:

Sitio principal y documentación de GitHub Actions



[Product](#) ▾ [Solutions](#) ▾ [Resources](#) ▾ [Open Source](#) ▾ [Enterprise](#) ▾ [Pricing](#)



[Sign in](#)

[Sign up](#)

[Features](#) / [GitHub Copilot](#) [Security](#) [Actions](#) [Codespaces](#) [Issues](#) [Code review](#) [Discussions](#) [Code search](#)

GitHub Actions

Automate your workflow from idea to production

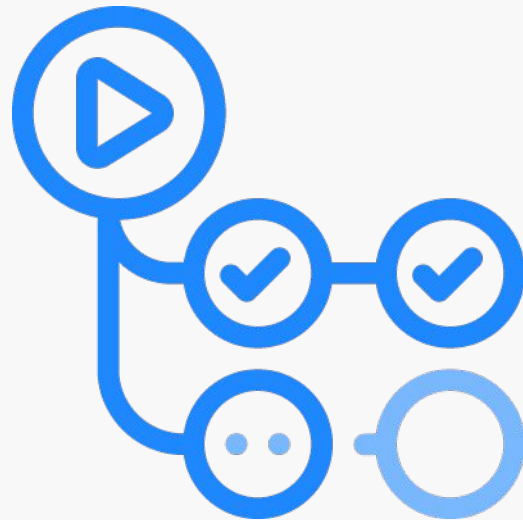
GitHub Actions makes it easy to automate all your software workflows, now with world-class CI/CD. Build, test, and deploy your code right from GitHub. Make code reviews, branch management, and issue triaging work the way you want.

[Get started with actions >](#)

[Contact sales >](#)

GitHub Actions

- Herramienta integrada para automatizar tareas dentro de GitHub
- Un **workflow** es un procedimiento automatizado que consiste en uno o más **jobs** gatillados por un evento, y pueden ser usados para:
 - hacer *builds* (ejecutables),
 - ejecutar tests,
 - empaquetar y publicar código,
 - lanzar nuevas versiones,
 - o hacer *deploy*



Importante:

Todo el código de esta clase se encuentra implementado en [IIC2513/Mi-primer-GA](#)

Se recomienda seguir los pasos descritos de manera local y solo consultar la implementación al final o en caso de dudas (*hint: ver los commits*)

GitHub Actions: Primera acción corriendo

1. Crea un repositorio solo con README y clonar localmente ([docs](#))
2. Crear `.github/workflows/demo.yml`
3. Hacer *push* del código a GitHub y ver pestaña Actions en el repositorio
4. La acción recién subida debería ser exitosa y podremos ver el *output*

Basado en [Quickstart for GitHub Actions \(GitHub docs\)](#)

```
name: Demo
run-name: Primera GitHub Action de ${ github.actor } ✨
on: [push]
jobs:
  Mi-primer-GitHub-Action:
    runs-on: ubuntu-latest
    steps:
      - run: echo "🚀 Evento de tipo ${ github.event_name }."
      - run: echo "💡 Job corriendo en ${ runner.os } en GitHub!"
      - run: echo "🌐 Rama ${ github.ref } de ${ github.repository }."
      - name: Clonando desde el repositorio
        uses: actions/checkout@v4
      - run: echo "💡 El repo ${ github.repository } se clona en runner."
      - run: echo "💻 Workflow listo para ejecutar el runner."
      - name: Listando los contenidos del repositorio
        run: |
          ls ${ github.workspace }
      - run: echo "🟢 El status de este job es ${ job.status }."
```

GitHub Actions: Primera acción corriendo

1. Crea un repositorio solo con README y clonar localmente ([docs](#))
2. Crear `.github/workflows/demo.yml`
3. **Hacer *push* del código a GitHub y ver pestaña Actions en el repositorio**
4. La acción recién subida debería ser exitosa y podremos ver el *output*

Basado en [Quickstart for GitHub Actions \(GitHub docs\)](#)



GitHub Actions: Primera acción corriendo

1. Crea un repositorio solo con README y clonar localmente ([docs](#))
2. Crear `.github/workflows/demo.yml`
3. Hacer *push* del código a GitHub y ver pestaña Actions en el repositorio
4. **La acción recién subida debería ser exitosa y podremos ver el *output***

Basado en [Quickstart for GitHub Actions \(GitHub docs\)](#)



Primera GitHub Action de aaossa ✨

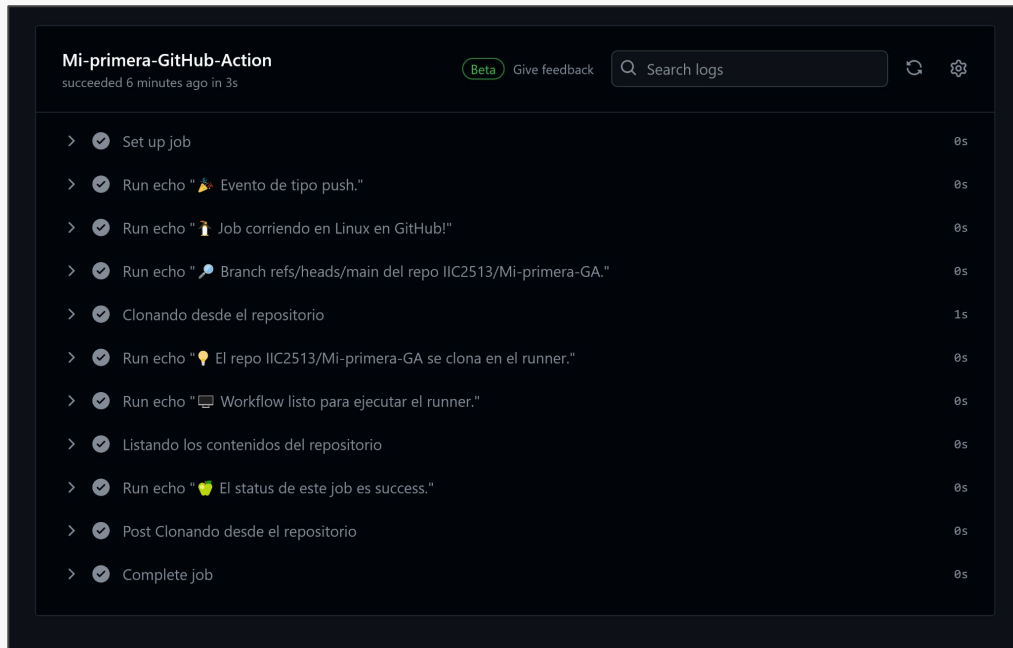
Demo #1: Commit 4be14a8 pushed by aaossa

main

GitHub Actions: Primera acción corriendo

1. Crea un repositorio solo con README y clonar localmente ([docs](#))
2. Crear `.github/workflows/demo.yml`
3. Hacer *push* del código a GitHub y ver pestaña Actions en el repositorio
4. **La acción recién subida debería ser exitosa y podremos ver el *output***

Basado en [Quickstart for GitHub Actions \(GitHub docs\)](#)



GitHub Actions: Acción con *linter*

1. En el mismo repositorio local, crear proyecto con yarn y agregar ESLint:
`yarn init -y`
`yarn add eslint --dev`
2. Crear `.gitignore`
3. Crear archivo de configuración de ESLint
4. Crear archivo `src/index.js`
5. Crear `.github/workflows/eslint.yml`
6. Hacer *push* del código a GitHub y revisar la pestaña Actions en el repositorio
7. La acción recién subida debería fallar

```
{  
  "name": "Mi-primer-GA",  
  "version": "1.0.0",  
  "main": "index.js",  
  "repository": "https://github.com/user/repo.git",  
  "author": "Antonio Ossa-Guerra <aaossa@uc.cl>",  
  "license": "MIT",  
  "devDependencies": {  
    "eslint": "^9.0.0"  
  }  
}
```

GitHub Actions: Acción con *linter*

1. En el mismo repositorio local, crear proyecto con yarn y agregar ESLint:
`yarn init -y`
`yarn add eslint --dev`
2. **Crear .gitignore**
3. Crear archivo de configuración de ESLint
4. Crear archivo `src/index.js`
5. Crear `.github/workflows/eslint.yml`
6. Hacer *push* del código a GitHub y revisar la pestaña Actions en el repositorio
7. La acción recién subida debería fallar

```
# .gitignore  
  
# Ignorar carpetas  
# de dependencias  
node_modules/
```


GitHub Actions: Acción con *linter*

1. En el mismo repositorio local, crear proyecto con yarn y agregar ESLint:
`yarn init -y`
`yarn add eslint --dev`
2. Crear `.gitignore`
3. **Crear archivo de configuración de ESLint**
4. Crear archivo `src/index.js`
5. Crear `.github/workflows/eslint.yml`
6. Hacer *push* del código a GitHub y revisar la pestaña Actions en el repositorio
7. La acción recién subida debería fallar

```
// eslint.config.js
module.exports = [
  {
    rules: {
      semi: 'error',
      'prefer-const': 'error',
    },
  },
];
```

GitHub Actions: Acción con *linter*

1. En el mismo repositorio local, crear proyecto con yarn y agregar ESLint:
`yarn init -y`
`yarn add eslint --dev`
2. Crear `.gitignore`
3. Crear archivo de configuración de ESLint
4. **Crear archivo `src/index.js`**
5. Crear `.github/workflows/eslint.yml`
6. Hacer *push* del código a GitHub y revisar la pestaña Actions en el repositorio
7. La acción recién subida debería fallar

```
// src/index.js

let a = 3;
console.log(a);
```

GitHub Actions: Acción con *linter*

1. En el mismo repositorio local, crear proyecto con yarn y agregar ESLint:
`yarn init -y`
`yarn add eslint --dev`
2. Crear `.gitignore`
3. Crear archivo de configuración de ESLint
4. Crear archivo `src/index.js`
5. **Crear `.github/workflows/eslint.yml`**
6. Hacer *push* del código a GitHub y revisar la pestaña Actions en el repositorio
7. La acción recién subida debería fallar

```
name: Linter
run-name: Ejecutando ESLint ✨
on: [push]
jobs:
  Ejecutando-ESLint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install modules
        run: yarn
      - name: Run ESLint
        run: yarn run eslint .
```

GitHub Actions: Acción con *linter*

1. En el mismo repositorio local, crear proyecto con yarn y agregar ESLint:
`yarn init -y`
`yarn add eslint --dev`
2. Crear `.gitignore`
3. Crear archivo de configuración de ESLint
4. Crear archivo `src/index.js`
5. Crear `.github/workflows/eslint.yml`
6. Hacer *push* del código a GitHub y revisar la pestaña Actions en el repositorio
7. **La acción recién subida debería fallar**



Ejecutando ESLint ✨

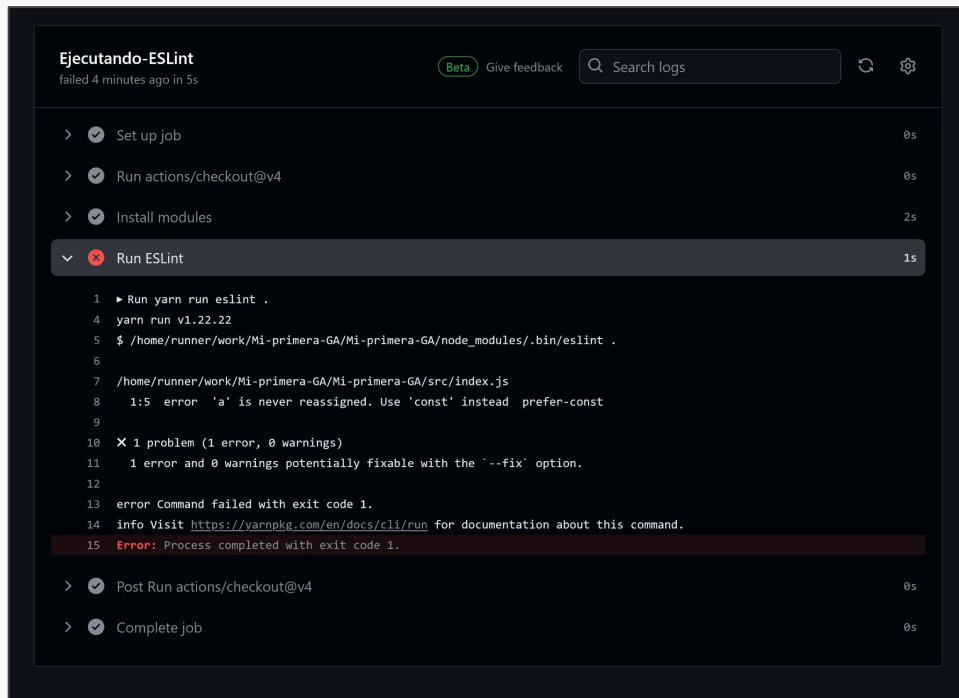


Linter #1: Commit [fd13698](#) pushed by aaossa

main

GitHub Actions: Acción con *linter*

1. En el mismo repositorio local, crear proyecto con yarn y agregar ESLint:
`yarn init -y`
`yarn add eslint --dev`
2. Crear `.gitignore`
3. Crear archivo de configuración de ESLint
4. Crear archivo `src/index.js`
5. Crear `.github/workflows/eslint.yml`
6. Hacer *push* del código a GitHub y revisar la pestaña Actions en el repositorio
7. La acción recién subida debería fallar



GitHub Actions: Arreglando según *linter*

1. Arreglar el código localmente desde la consola usando `--fix`
`yarn run eslint . --fix`
2. Hacer *push* del código a GitHub y revisar la pestaña Actions en el repositorio
3. La acción recién ejecutada debería completarse exitosamente

```
diff --git a/src/index.js b/src/index.js
index 400d0cc..cfb95c1 100644
--- a/src/index.js
+++ b/src/index.js
@@ -1,2 +1,2 @@
-let a = 3;
+const a = 3;
  console.log(a);
```

GitHub Actions: Arreglando según *linter*

1. Arreglar el código localmente desde la consola usando `--fix`
`yarn run eslint . --fix`
2. Hacer *push* del código a GitHub y revisar la pestaña Actions en el repositorio
3. **La acción recién ejecutada debería completarse exitosamente**



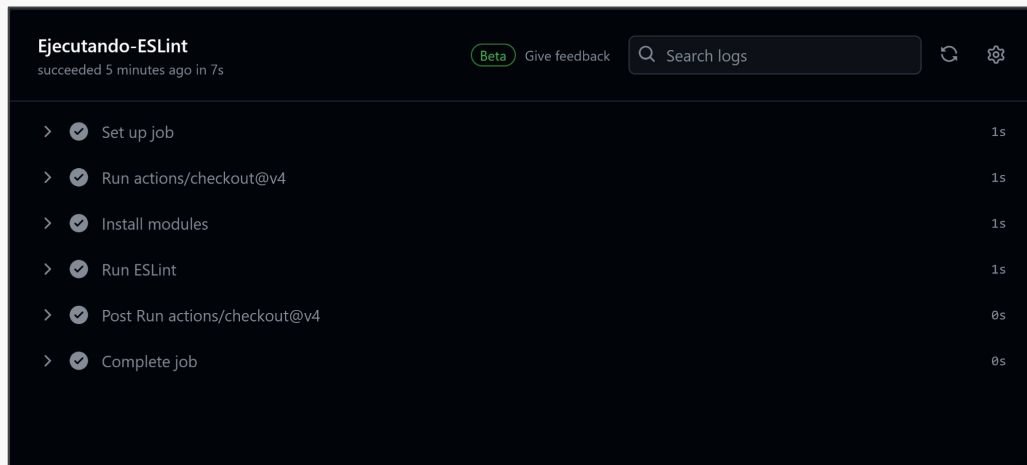
Ejecutando ESLint ✨

Linter #2: Commit 1d5cadb pushed by aaossa

main

GitHub Actions: Arreglando según *linter*

1. Arreglar el código localmente desde la consola usando `--fix`
`yarn run eslint . --fix`
2. Hacer *push* del código a GitHub y revisar la pestaña Actions en el repositorio
3. La acción recién ejecutada debería completarse exitosamente



curl:

Sitio principal y documentación de curl



[Download](#)

[Documentation](#)

[libcurl](#)

[Get Help](#)

[Development](#)

[News](#)



command line tool and library
for transferring data with URLs
(since 1998)

What is curl used for?

curl is used in command lines or scripts to transfer data. curl is also **libcurl**, used in cars, television sets, routers, printers, audio equipment, mobile phones, tablets, medical devices, settop boxes, computer games, media players and is the Internet transfer engine for countless software applications in over *twenty billion installations*.

curl is used daily by virtually every Internet-using human on the globe.

curl supports

Top Sponsors



wolfSSL

Agenda

Calidad de software

QA: Aseguramiento de calidad

CI: *Continuous Integration*



CD: *Continuous Deployment*

Continuous Deployment (CD)

- Entrega Continua es una metodología en la que mantenemos la capacidad de cambiar la versión de una aplicación en cualquier momento
- Busca reducir el tiempo de entrega y mejorar la reproducibilidad del proceso de *deployment*, haciendo el proceso de manera no-manual
- Es buena práctica automatizar este proceso con herramientas como:
 - Ansible
 - Plataformas incluyen integraciones para automatizar esto

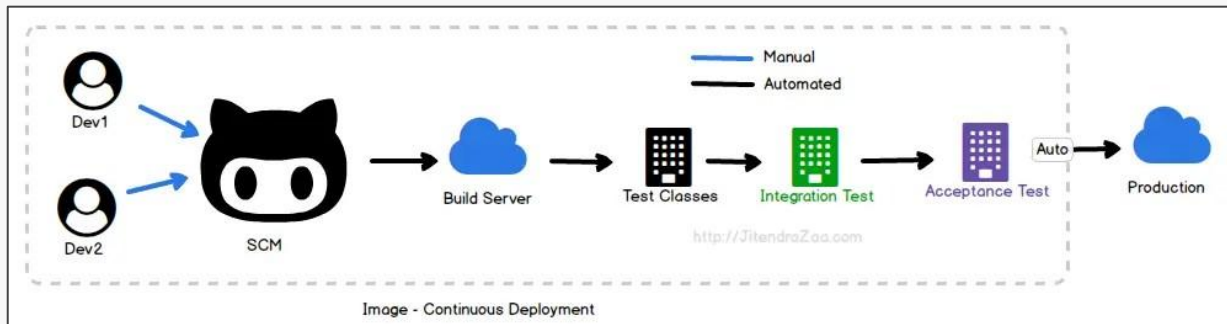


Figura: [Continuous Integration vs. Continuous Delivery vs. Continuous Deployment – Jitendra Zaa](#)

Continuous Deployment (CD)

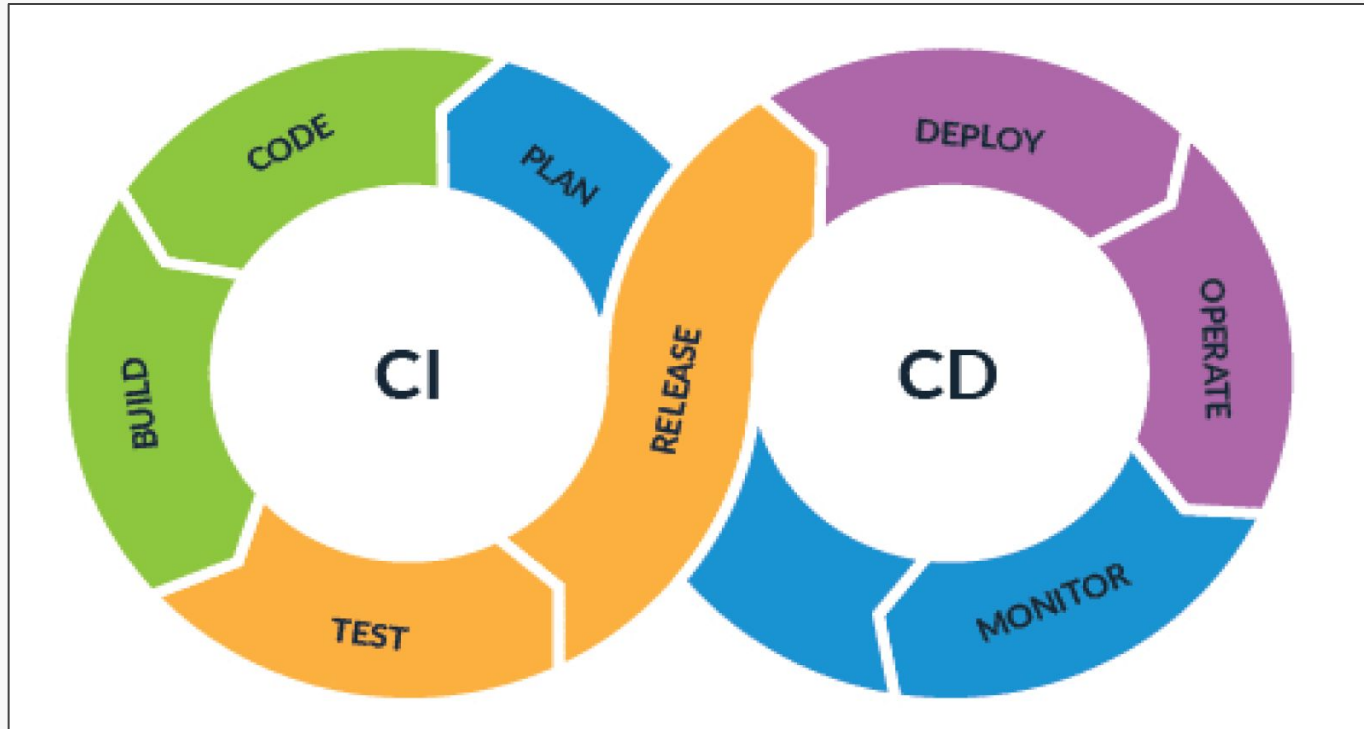


Figura: [*Continuous Integration and Delivery in an Agile Environment - Stridefuture*](#)

[GitHub Pages](#):

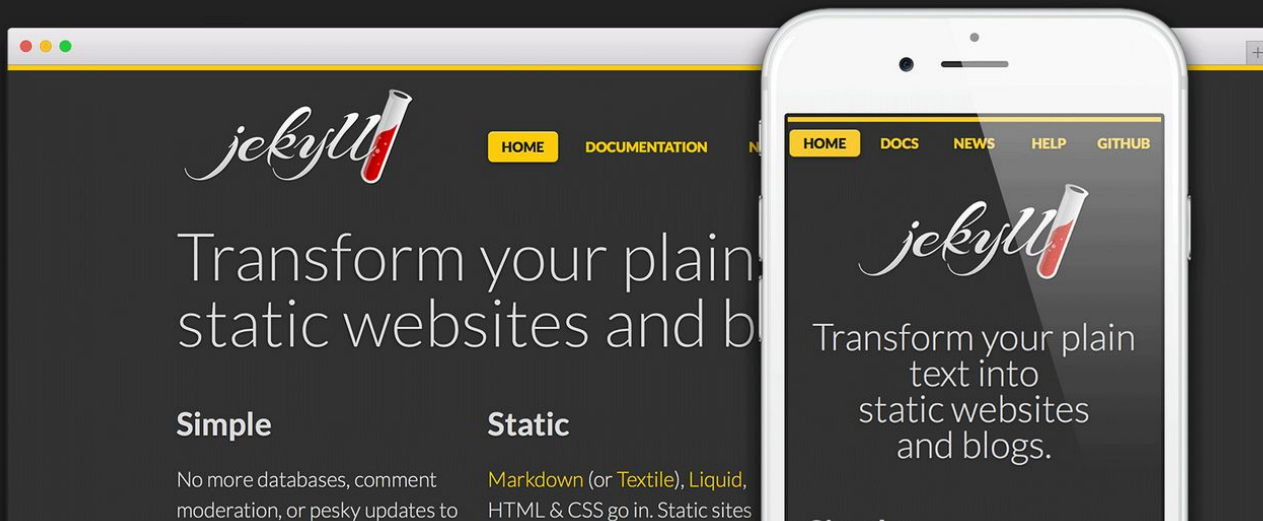
Sitio principal y documentación de GitHub Pages

Pages Help

GitHub Pages

Websites for you and your projects.

Hosted directly from your [GitHub repository](#). Just edit, push, and your changes are live.



Render:

Hosting para aplicaciones cloud



[Product](#)

[Pricing](#)

[Blog](#)

[Docs](#)

[Changelog](#)

[Careers](#)

[About](#)

[Sign In](#)

[Get Started](#)

Your fastest path to production

Build, deploy, and scale your apps with unparalleled ease –
from your first user to your billionth.

[Get Started for Free >](#)

[Contact Sales >](#)



Netlify:

Hosting para aplicaciones cloud



Platform ▾ Solutions ▾ Integrations Start Building ▾ Docs Pricing



Contact

Log in

Sign up

Push your ideas to the web

Deploy any modern frontend stack, from marketing sites to AI apps.
Join millions of developers and teams shipping faster on Netlify.

Get started

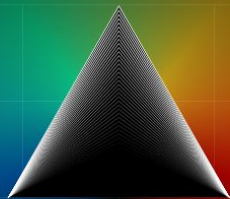
Request a demo

Vercel: Hosting para aplicaciones cloud

[Products ▾](#)[Solutions ▾](#)[Resources ▾](#)[Enterprise](#)[Docs](#)[Pricing](#)[Log In](#)[Contact](#)[Sign Up](#)

Your complete platform for the web.

Vercel provides the developer tools and cloud infrastructure
to build, scale, and secure a faster, more personalized web.

[▲ Start Deploying](#)[Get a Demo](#)

Agenda

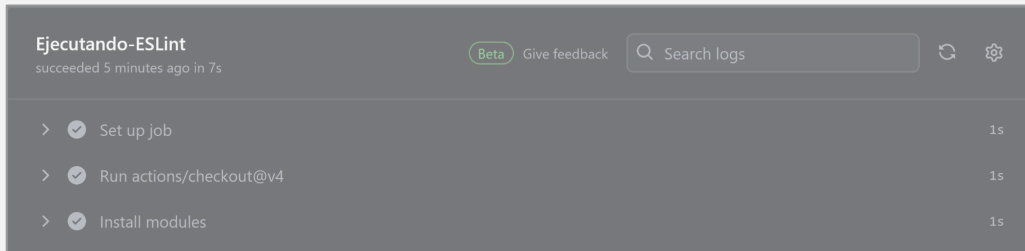
Calidad de software



Actividad 3

GitHub Actions: Arreglando según *linter*

1. Arreglar el código localmente desde la consola usando `--fix`
`yarn run eslint . --fix`
2. Hacer *push* del código a GitHub y revisar la pestaña Actions en el



Actividad 3#Implementando GitHub Actions

Desafío práctico: GitHub Actions

1. Creen un repositorio especial para este desafío. Lo podrán crear con la siguiente invitación de GitHub Classroom:

<https://classroom.github.com/a/F9tulGTE>

2. Mediante GitHub Actions, deben realizar las acciones detalladas en las instrucciones del README del repositorio asignado;
 - Revisar el estilo del código usando `eslint`
 - Realizar un POST en la url: <https://pokemon-api-vpmj.onrender.com/cards> con exactamente el mismo *body* que se indica en el README
 - Actualizar la versión indicada en el README
 - Actualizar el valor del *timestamp* del README que indica la última ejecución

Agenda

Calidad de software

Actividad 3

✨ **¿Qué se viene pronto?**

Próximas fechas importantes: clases y ayudantías

[illegible]

[illegible]

Clase 10

QA, CI & CD

IIC2513 - Tecnologías y Aplicaciones Web

Antonio Ossa Guerra
aaossa@ing.puc.cl